

SEUPD@CLEF: Team RETRIX on Source Retrieval for Scientific Web Claims

Notebook for the CheckThat! Lab at CLEF 2026

Fabio Baldan¹, Davide Donati¹, Alvisè Garberino¹, Giancarlo Padoan¹ and Marco Tessari¹

¹University of Padua, Italy

Abstract

We present the results of the RETRIX group for the CheckThat! Lab at CLEF 2026 Task 1, whose goal is to identify the scientific paper implicitly referenced in a social media post. The input consists of short, noisy and usually informal posts that mention scientific findings without providing explicit links or citations. Our pipeline first translates non-English posts into English, then expands them using an embedding model and terms extracted from the top five documents ranked by similarity. We then apply a hybrid retrieval system that combines dense FAISS search with sparse lexical matching and ColBERT multi-vector scoring ($s = 0.5 s_{\text{dense}} + 0.15 s_{\text{sparse}} + 0.35 s_{\text{colbert}}$). Finally, we perform neural reranking with a fine-tuned cross-encoder model.

Keywords

Fact Checking, Information Retrieval, Scientific Document Retrieval, Social media fact verification, CLEF, Neural Re-ranking, Cross Encoder

1. Introduction

The rapid spread of scientific claims through social media creates a difficult fact-checking scenario: posts often report, simplify or sensationalize research findings while omitting the original source. In many cases, the cited study is not linked explicitly with a URL or a bibliographic reference but only mentioned indirectly through fragments such as the reported finding, the venue, the institution or the authors. As a consequence, verifying such claims requires first identifying the scientific publication that is being implicitly referenced.

This problem is the focus of Task 1 of the CLEF 2026 CheckThat! Lab, *Source Retrieval for Scientific Web Claims*: given a social media post containing a scientific claim and an implicit reference to a paper, the goal is to retrieve the correct publication from a fixed collection of 10,000 candidate papers written in English.

The task is challenging for several reasons. First, the linguistic gap between social media and scientific writing is substantial: posts are short, informal and often contain paraphrases, slogans, hashtags or incomplete statements whereas scientific papers use more precise and technical language. Second, the reference to the source paper is implicit rather than explicit so the retrieval system must infer the connection from partial semantic evidence rather than from direct citations. Third, relevant clues may appear in different forms, such as mentions of a scientific result, a medical outcome, a research institution or a publication venue, requiring the system to combine lexical and semantic evidence effectively.

An additional challenge is multilinguality: the official query sets include English, German and French posts while the candidate pool remains shared across languages.

Team RETRIX approaches this problem with a four-stage sequential pipeline.

“Search Engines”, course at the master degree in “Computer Engineering”, Department of Information Engineering, and at the master degree in “Data Science”, Department of Mathematics “Tullio Levi-Civita”, University of Padua, Italy. Academic Year 2025/2026

✉ fabio.baldan.1@studenti.unipd.it (F. Baldan); davide.donati@studenti.unipd.it (D. Donati);
alvisè.garberino@studenti.unipd.it (A. Garberino); giancarlo.padoan@studenti.unipd.it (G. Padoan);
marco.tessari.8@studenti.unipd.it (M. Tessari)



© 2026 Copyright for this paper by its authors.

Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

1. In a preliminary stage we translate to English all the queries written in French and German using the model `utter-project/EuroLLM-9B-Instruct-2512` [1], specifically funded by the EU for understanding and generating text in all European languages. This allows us to handle all queries the same way, independently of the original language and consequently of the semantic and structural differences.
2. Raw tweet-style queries are cleaned of emojis and symbols like "#" and "@" and then enriched via pseudo-relevance feedback: `BAAI/bge-large-en-v1.5` [2] encodes both queries and corpus documents into a shared embedding space, retrieves the top-5 semantically similar papers per query and extracts the most frequent terms from those papers as expansion tokens.
3. The expanded queries are fed into `BAAI/bge-m3` [3] operating in hybrid mode, which combines dense inner-product search via FAISS [4] with sparse lexical matching and ColBERT multi-vector scoring to produce an initial ranked list of candidates.
4. Finally, the top candidates are re-ranked by a neural cross-encoder, in our case `nvidia/llama-nemotron-rerank-1b-v2` [5], fine-tuned on the English training dataset of the CLEF CheckThat! Task 1 2026 and all the queries' datasets of the 2025 CLEF CheckThat! SubTask 4b.

The paper is organized as follows: Section 2 discusses the most relevant research directions related to this task; Section 3 describes in more detail our approach; Section 4 explains our experimental setup; Section 5 discusses our main findings; finally, Section 6 draws some conclusions and outlooks for future work.

2. Related Work

The task addressed in this paper lies at the intersection of fact-checking and information retrieval. A natural starting point is lexical retrieval where probabilistic ranking models such as BM25 remain strong baselines thanks to their efficiency, interpretability and robustness in candidate generation [6]. In the setting of scientific claim source retrieval, lexical matching is especially useful when a post mentions recognizable entities or terminology that also appears in the title, abstract, venue or author list of the referenced paper.

Neural retrieval methods have been proposed to mitigate the lexical mismatch between queries and documents by learning dense semantic representations. More broadly, neural information retrieval has shifted attention from exact token overlap toward representation learning and learned matching functions [7]. Within this line of work, dense dual-encoder architectures such as Dense Passage Retrieval (DPR) demonstrated that learned query and document embeddings can be highly effective for first-stage retrieval [8]. This is especially relevant to our task because social media posts often paraphrase scientific findings rather than reuse the exact wording found in the original publication. Building on this direction, BGE-M3 unified dense, sparse, and multi-vector retrieval in a single multilingual model [3], enabling hybrid scoring that combines embedding similarity with lexical token weights, a combination particularly well-suited to cross-lingual settings where queries and documents share few surface tokens.

Multilingual information retrieval presents additional challenges beyond monolingual settings, as systems must bridge language mismatches before relevance can be estimated. Machine translation provides one practical solution, but it can introduce errors and vocabulary drift, especially for technical or domain-specific terminology [9]. Cross-lingual retrieval systems must also normalize text across different writing systems and grammatical structures. In this context, query translation followed by retrieval in a shared multilingual embedding space has emerged as a practical approach, though translation errors can compound retrieval errors and substantially affect downstream performance.

Vocabulary mismatch between short, informal queries and longer scientific documents can be further reduced through query expansion. Classical pseudo-relevance feedback methods such as RM3 [10] address this issue by re-weighting query terms using top-ranked documents from an initial retrieval pass. Neural extensions of this paradigm replace or complement lexical evidence with dense representations [11]. Following this general approach, our pipeline incorporates embedding-based query expansion to enrich query representations before scoring against the corpus.

Between initial retrieval and final ranking, many modern systems adopt multi-stage pipelines to balance efficiency and effectiveness [7]. Within these pipelines, late-interaction models such as ColBERT preserve fine-grained token-level matching while remaining substantially more scalable than fully pairwise neural scoring over the whole collection [12]. Similarly, transformer-based re-rankers have become a common second-stage component for refining an initial candidate set through stronger query-document interactions [13]. Taken together, these approaches motivate retrieval architectures that first generate a manageable set of candidates and then apply more expensive relevance estimation only where it is most useful.

Recent work has argued that reranker fine-tuning benefits from hard negatives because they force the model to learn finer-grained relevance boundaries than easy or random negatives provide [14]. A practical instantiation of this strategy trains the reranker on negatives mined from the same retrieval model whose errors it is meant to correct, yielding a form of bootstrap self-improvement. Adapting large language models to retrieval tasks is further facilitated by parameter-efficient fine-tuning with LoRA [15], which has been shown to match or approach full fine-tuning quality for passage reranking while substantially reducing GPU memory requirements [16]. This perspective is also consistent with recent practical guidance for training cross-encoder rerankers with Sentence Transformers [17]. In that setting, fast embedding-based retrieval is used to build a candidate pool, after which the cross-encoder jointly scores query-document pairs and benefits from negatives mined from nearby embedding neighborhoods rather than from trivial mismatches.

Our work is also related to the broader literature on automatic claim verification and more specifically to the CheckThat! shared tasks, which have addressed the detection, verification, and analysis of claims in social media across several editions [18, 19, 20]. Recent participants in the same task line have independently converged on hybrid retrieval plus reranking as the dominant pipeline configuration [21, 22], confirming the high-level design choices taken.

3. Methodology

We started from a lexical baseline based on Apache Lucene BM25 and progressively moved to a multi-stage neural pipeline. Early experiments focused on analyzer choices, stopword lists, and BM25 variants. These runs were useful for establishing a stable baseline, but they also showed that lexical overlap alone is insufficient for this task.

We adopted a four-stage pipeline (Figure 1) for the proposed information retrieval system, consisting of:

- **Query translation:** queries in French or German are first translated into English.
- **Query expansion:** we expanded the English query with additional context to bridge the gap between informal and domain specific language.
- **Retrieval:** we retrieved relevant documents from the corpus using the expanded query.
- **Reranking:** we reranked the retrieved documents based on their relevance to the original query.

For each of these stages, we experimented with different models and kept the ones that performed best. The following sections explain the four stages in detail.

3.1. Query translation

In the preliminary stage of our pipeline, non-English queries (French and German) are translated into English. This is done query by query using the model `utter-project/EuroLLM-9B-Instruct-2512` with the following prompt:

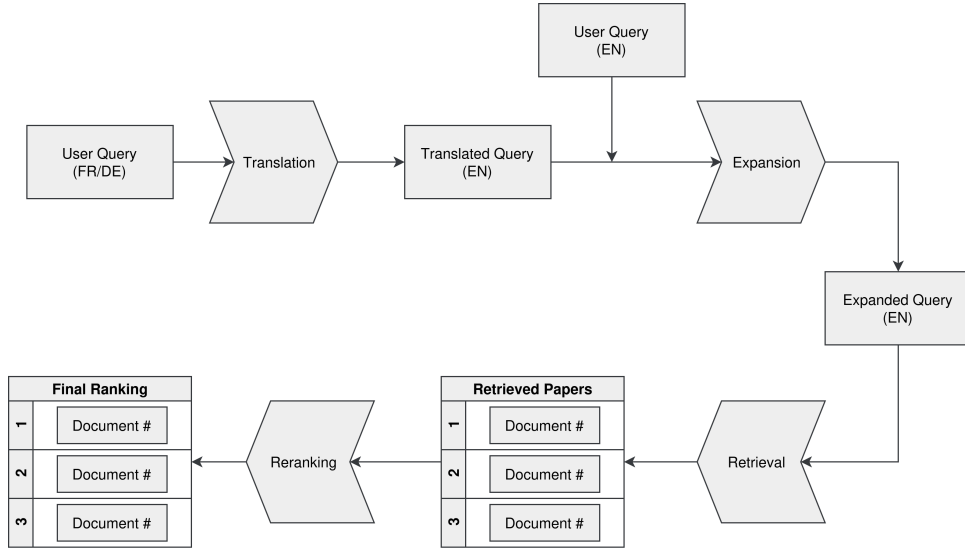


Figure 1: Four-stage pipeline for information retrieval.

LLM Prompt

system: You are a professional translator.
 Translate the user’s text to English.
 Output ONLY the English translation.
 Do not include the original text, explanations, labels, or any other content.

user: Translate this {French/German} text to English.
 Output only the English translation, nothing else.

{Query original text x }

Given x as the original query text, the model predicts probabilities for the next token with [23]:

$$P(y_t = i \mid y_{<t}, x) = \text{softmax}(z_t/T)_i$$

With our model configuration, the prediction becomes greedy, following [1]:

$$y_t = \arg \max_i P(y_t = i \mid y_{<t}, x)$$

The resulting English queries are then fed into the following step of our pipeline.

3.2. Query Expansion

Following the translation stage, each query is processed by a deterministic query-expansion module designed to reduce query–document vocabulary mismatch between short user claims and the terminology used in the scientific corpus. The module does not rely on generative LLM-based paraphrasing. Instead, it combines lexical feature extraction with dense semantic retrieval and neighborhood-based term induction. This design is consistent with the established role of query expansion in mitigating vocabulary mismatch in information retrieval and with the use of embedding models for semantic search.

- The translated English query is first analyzed with spaCy’s `en_core_web_sm` pipeline, which provides token-level linguistic annotations, including stop-word indicators and coarse-grained part-of-speech labels from the Universal POS tag set. Candidate lexical anchors are then identified by retaining tokens tagged as NOUN or PROPEN while excluding stop words and tokens shorter than three characters. The resulting set is deduplicated to produce a compact lexical signature that captures the principal nominal content of the query.

- The semantic expansion stage operates over a corpus in which each document is represented as the concatenation of its title and abstract. These title–abstract pairs are embedded with BAAI/bge-large-en-v1.5. In accordance with the documented BGE retrieval interface, query embeddings are produced by prepending the instruction `{Represent this sentence for searching relevant passages:}` to the translated query, whereas the corpus documents are encoded as retrieval targets. Similarity is then computed on normalized embeddings using the dot product. For unit-length vectors, dot-product scoring is equivalent to cosine similarity while being computationally more efficient.

$$s(q, d) = e_q^\top e_d = \sum_i e_{q,i} e_{d,i}$$

Because embeddings $e = \frac{v}{\|v\|_2}$ are L2-normalized $e_q^\top e_d = \cos(q, d)$ [24].

The five highest-scoring documents are retained as the local semantic neighborhood used for expansion.

In the described implementation, candidate expansion terms are extracted from the retrieved documents after lowercasing, ASCII normalization, mention and hashtag removal, stop-word filtering, and exclusion of short tokens. The candidate pool is aggregated across the retrieved neighborhood and assigned coarse weights derived from the semantic similarity of the source documents, so that terms originating from more similar neighbors contribute more strongly. Terms already present in the query are removed and the fifteen highest-ranking candidates are retained as explicit expansion terms.

$$K = \text{unique}\{\ell(t) : POS(t) \in \{\text{NOUN}, \text{PROPN}\}, \neg stop(t), |t| > 2\}$$

$$w_h = \max(1, \text{round}(5s_h))$$

$$c(t) = \sum_h w_h tf(t, d_h)$$

The final expanded query is constructed by concatenating three components: the cleaned translated query, the noun and proper-noun-based keyword set extracted directly from the query and the expansion terms induced from the retrieved document neighbourhood. This output expanded query is then used by the downstream retrieval pipeline.

Before settling on this query expansion strategy, we also tried two LLM models to see if they could improve our performance, `meta-llama/Llama-3.2-1B` and `openai/gpt-oss-20b`, with the following prompt:

LLM Prompt

```
system: "Reasoning: {reasoning_effort}"
You are a strict JSON-only retrieval query expansion generator.
Return the final answer only as valid JSON.

user: You generate retrieval-specific query expansions.
Return only one valid JSON object with exactly these string fields:
{
  "keywords": "...",
  "sparse": "...",
  "embedding": "...",
  "colbert": "...",
  "expanded": "..."
}

Rules:
- Do not include markdown, code fences, emoji, commentary, arrays, or nested objects.
- All five values must be strings.
- Do not hallucinate facts.
- Do not translate unless translation is already present in the query.
- Preserve original-language terms.
- keywords: compact important terms only; entities, nouns, technical terms, names, locations, dates, identifiers, acronyms, modifiers; no sentence.
- sparse: lexical BM25/SPLADE-style expansion; exact-match terms, synonyms, acronyms, aliases, spelling and morphology variants; phrase fragments separated by spaces or semicolons; no long prose.
- embedding: concise HyDE/query2doc-style natural-language semantic pseudo-document; include intent, context, and likely relevant-document framing.
- colbert: concise late-interaction expansion; preserve important tokens and entities; add focused context and aliases; avoid synonym spam.
- expanded: readable general-purpose merged expansion string combining the useful hints from the other fields.
```

This approach, which used a different expansion for each retrieval representation, did not improve performance. We therefore retained the simpler and more effective configuration based on a single expanded query. Section 3.3 describes in more detail how these expansion strategies were evaluated during retrieval.

3.3. Retrieval

The retrieval stage is designed as a high-recall candidate-generation step between query expansion and neural reranking. Its objective is not to make the final relevance decision but to ensure that the true source paper is included in a sufficiently large candidate set for the subsequent, more computationally expensive cross-encoder stage. For this reason, after experimenting with a Lucene BM25 baseline, BGE-M3 using dense vectors only and other bi-encoder models, the final pipeline adopts **BAAI/bge-m3** in a hybrid retrieval setting.

This choice reflects the nature of the task. Social media claims often contain informal paraphrases, hashtags, named entities, numerical values and fragments of scientific terminology. Therefore, relying exclusively on exact lexical overlap or exclusively on dense semantic similarity may discard useful evidence. Hybrid retrieval is better suited to this setting because it combines complementary retrieval signals and can offer higher accuracy and stronger generalization capabilities [3].

The retriever searches the fixed paper collection provided by the task organizers. Each paper is represented at document level by concatenating its title and abstract and the retrieved unit is always the paper pubkey. We did not split papers into smaller chunks because the task requires identifying the source publication rather than retrieving a specific evidence passage. Using one representation per

paper also avoids an additional chunk-to-document aggregation step. Although the collection includes metadata such as venue and authors, this hybrid retrieval stage uses only titles and abstracts to speed up the inference time.

All input queries are converted into plain English, either because they are originally written in English or because they are translated before retrieval, see Section 3.1. They are then expanded using the procedure described in Sections 3.2.

We chose **BAAI/bge-m3** because it is well suited to this retrieval setting. The model can produce dense vectors, sparse lexical weights and ColBERT-style multi-vector representations from the same encoder.

This allows the retriever to combine broad semantic matching, exact term evidence and fine-grained token-level matching without switching between unrelated encoders.

For the dense representation, the document or query embedding is obtained from the normalized CLS representation [3]:

$$e = \text{norm}(H[0])$$

For the sparse representation, BGE-M3 assigns lexical weights to tokens [25]:

$$w_{qt} = \text{ReLU}(W_{lex}^\top H_q[i])$$

The sparse similarity between a query and a paper is then computed over overlapping terms:

$$s_{lex} = \sum_{t \in q \cap p} w_{qt} w_{pt}$$

Candidate selection is performed in two steps. First, all documents are encoded as dense BGE-M3 vectors and searched with an inner-product FAISS index [4]. The dense FAISS score is computed using inner product [26]:

$$s_{IP}(q, d) = q^\top d$$

Since the dense vectors are normalized, this inner product is equivalent to cosine similarity:

$$q^\top d = \cos(q, d)$$

This dense stage acts as an efficient broad filter over the whole collection. The default setting retrieves a large candidate pool of `candidate_multiplier` $\times$ `top_k`, with `candidate_multiplier` = 3 and `top_k` = 2000, before the final truncation. The number of candidates passed to the hybrid rescoring stage is:

$$K_c = \min(N, \max(K, Km))$$

This deliberately large candidate set favours recall, which is more important than precision at this stage.

Second, the dense candidates are rescored using the sparse and ColBERT-style representations. The sparse component rewards important overlapping terms, while the ColBERT-style component captures local token correspondences that may be lost when each document is represented by a single dense vector [12]. The ColBERT-style late-interaction score is computed as [12]:

$$s_{colbert}(q, d) = \frac{1}{N_q} \sum_{i=1}^{N_q} \max_{1 \leq j \leq N_d} q_i^\top d_j$$

The final configuration was selected through a series of development runs. We first tuned the lexical baseline by varying the Lucene analyzer, stop-word lists, stemming strategy and BM25 searcher variants. These experiments improved the classical baseline but also highlighted its limitations for this task: even with query expansion and reranking, BM25-based candidate generation remained more dependent on surface-level lexical overlap than desired.

We then evaluated dense retrieval with several bi-encoder models. Among these, **BAAI/bge-m3** achieved the strongest performance and improved the semantic coverage of the candidate set, particularly for paraphrased and multilingual claims. We also tested BGE-M3 in a hybrid retrieval setting, using a setup that differed slightly from the final configuration. In those experiments, sparse, dense, and ColBERT-style retrieval were performed using dedicated query expansions tailored to each retrieval mode. More specifically, each query was represented through three complementary views: an embedding view for semantic matching, a sparse view for lexical and term-based evidence, and a ColBERT view for fine-grained token-level interaction.

However, the best results in our analysis were obtained with a simpler configuration in which BGE-M3 hybrid retrieval used the same expanded query, described in the previous section, for all three retrieval modes. We then tested several weighting configurations. Among them, the run labelled `bge_m3_hybrid_513` was retained because it provided the strongest recall-oriented trade-off among the retrievers evaluated without reranking. This configuration used weights of 0.5, 0.15 and 0.35 for dense, sparse, and ColBERT scoring, respectively, with a multiplier of 3 and a maximum hybrid input length of 384.

A closely related configuration with a larger ColBERT weight achieved a nearly identical MRR@5 but a lower Recall@100. We therefore preferred the configuration that better matched the role of retrieval as a candidate generator. This choice was further confirmed by downstream experiments: reranking candidates from `bge_m3_hybrid_513` consistently outperformed the corresponding BM25-based reranked runs. In addition, increasing the reranked candidate depth from a few hundred documents to between 1000 and 2500 documents increased the final recall by giving the neural reranker a larger candidate pool.

We also experimented with a learned fusion strategy for the hybrid signals. In particular, the experimental setup included a LightGBM LambdaMART option that builds features from the dense, sparse, and ColBERT scores, together with normalized score variants and dense-rank features, and then learns how to combine them. However, the retained retrieval configuration uses simpler linear fusion, which is easier to reproduce and aligns well with the recall-oriented role of this stage.

The final hybrid retrieval score is therefore computed as:

$$s(q, d) = 0.5 s_{\text{dense}}(q, d) + 0.15 s_{\text{sparse}}(q, d) + 0.35 s_{\text{colbert}}(q, d).$$

The largest weight is assigned to the dense score because it provides the main semantic signal. The sparse and ColBERT components add complementary lexical and fine-grained matching evidence. Candidates are sorted by this score in descending order and truncated to the final `top_k`. The retrieval output is a JSON mapping from each query ID to an ordered list of paper pubkeys. Scores are not exported, because the next stage treats retrieval as a candidate generator and recomputes relevance using a cross-encoder over the returned document identifiers.

3.4. Reranking

The reranking stage receives the candidate list produced by the hybrid retriever and reorders it using cross-encoder models. Its goal is to improve early precision, especially MRR@5, while preserving as much recall as possible from the first stage.

Cross-encoder scoring

Our strongest single-stage reranker is `nvidia/llama-nemotron-rerank-1b-v2`, a 1B-parameter LLaMA-based model fine-tuned as a sequence classifier. For each query-document pair, we build an input sequence using the official NVIDIA prompt template:

LLM Prompt

```
question:{query} \n \n passage:{passage}
```

The model computes a scalar logit $z \in \mathbb{R}$ through a linear classification head applied to the final hidden state, and the relevance score is obtained by applying a sigmoid:

$$s(q, d) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Documents are then sorted in descending order of $s(q, d)$. In practice, reranking deeper candidate pools (500–2500 documents) consistently produced better final results than shallower pools, confirming the importance of high recall at the retrieval stage to compensate for miss-labeling from weaker previous retrieval-stages. At this stage, queries are evaluated against the corpus of the documents taking into account their title, abstract and author.

We also evaluated alternative rerankers, including Qwen3 and GTE-ModernBERT variants

3.5. Fine-tuning the Reranker

To improve the Nemotron reranker on the domain-specific characteristics of the CheckThat! task, we fine-tuned `nvidia/llama-nemotron-rerank-1b-v2` using Parameter-Efficient Fine-Tuning (PEFT) with Low-Rank Adaptation (LoRA) on data from both the 2025 and 2026 editions of the task.

Training data and hard-negative mining

Training examples are query-document pairs with binary labels: a positive document is the gold source paper associated with the query, while negative documents are hard negatives mined automatically using `sentence-transformers/static-retrieval-mrl-en-v1` [27, 28, 29]. Hard negatives are selected from the range $[5, 50]$ nearest neighbors (by embedding similarity) that satisfy $\text{score} \leq 0.85$ and lie at least 0.05 below the positive score in cosine similarity (absolute margin) preventing trivially easy or falsely relevant negatives from polluting training that would under-mine the effectiveness of the fine-tuning.

Training data combines two years of the task:

- **2026 data:** split 90% training / 10% development. The development set is drawn exclusively from 2026 data for consistency with the official CT26 Task 1 evaluation protocol. Due to time-constraints we used just the `en_train` datasets but we suppose that also German and French translated queries datasets could improve the quality of the fine-tuning;
- **2025 data:** all available topics (dev, test-gold, and train splits) are used entirely for training, never for development.

This design ensures that the dev set used to track MRR@5 during training is distribution-aligned with the contest test queries. Each training record contains one query, one positive passage and up to five hard negatives.

LoRA architecture

To fine-tune the 1B-parameter base model with limited GPU memory, we apply LoRA [15] with rank $r = 16$ and scaling factor $\alpha = 32$. Adapters are inserted into all attention and feed-forward projection layers: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. The classification head (`score`) is included in `modules_to_save` and is fully updated during training. The task type is `SEQ_CLS`, matching the sequence-classification objective of the reranker.

Training objective

The model produces a single scalar logit z per input pair. Training minimizes binary cross-entropy with logits over all positive ($y = 1$) and negative ($y = 0$) pairs in each batch:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \left[y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i)) \right]$$

where B is the effective batch size and σ is the sigmoid function. Gradient accumulation over 4 steps is used to simulate a larger effective batch size of 32 without additional memory overhead.

Optimization

The model is trained with AdamW, a learning rate of 2×10^{-5} , a linear warmup over 10% of the total training steps, and gradient clipping at norm 1.0. Training runs for 3 epochs with bfloat16 mixed precision and gradient checkpointing enabled to reduce VRAM usage. The checkpoint with the best MRR@5 on the 2026 development set is retained.

Inference

At inference time, the LoRA adapter weights are merged into the base model via `merge_and_unload()` before scoring, so no PEFT overhead is incurred during evaluation. The scoring procedure and prompt format are identical to those of the base reranker (Section 3.4) and the fine-tuned model is used as a drop-in replacement in the reranking stage.

3.6. Evaluation

We evaluate systems on the CheckThat! Task 1 query sets by matching retrieved pubkeys against the gold pubkey for each query. The official optimization target is MRR@5, which we use as the primary score during model and parameter selection. In parallel, we track Recall@100 to monitor candidate-generation quality and to verify that reranking gains do not come from overly aggressive filtering.

For internal analysis, our evaluator also computes additional ranking metrics (e.g., Recall@1/5/10/100, Precision@1/5/10, MAP, and nDCG@k). This broader view is useful to interpret trade-offs between recall-oriented retrieval settings and precision-oriented reranking settings while keeping MRR@5 as the main criterion for reporting and model selection.

4. Experimental Setup

All experiments were conducted on our own machines, using a computer with an AMD Ryzen 9 5900X, 32 GB of RAM, and an NVIDIA RTX 3090 GPU (10,496 CUDA cores, 1.70 GHz boost clock, 24 GB of GDDR6X memory, 35.58 TFlops) for resource-intensive workloads and our laptops for standard workloads and program refinement. We used four laptops; the two most powerful are a MacBook Air M3 and a machine with no dedicated GPU, 16 GB of RAM, and an Intel Core i7-13700H. For reproducibility, all environment settings have been documented and are available in the Bitbucket repository at <https://bitbucket.org/upd-dei-stud-prj/seupd2526-retrix/src/master/> or can be reproduced by following the instructions in this paper. The task evaluation metric is MRR@5; in our runs, we also report Recall@100 to provide a more complete picture of the differences between configurations.

4.1. Dataset

The dataset consisted of 10,000 scientific publications that had to be paired with tweet-like posts written in three languages: German, French, and English. There are no explicit references such as URLs in any query; instead, each tweet contains an implicit reference to a scientific paper, for example through mentions of authors or keywords related to the paper. The dataset files are all in JSON format. The publications dataset contains the columns pubkey (unique identifier), title, abstract, venue/location, and authors, while the query datasets contain the columns index, pubkey (ID of the positive scientific paper that has to be matched with the query), and text.

Table 1
Dataset statistics by language

Split	Language	File	Queries		Length (tokens)			
			#	%	Min	Median	Max	Mean
Train	EN	en_train.json	14977	77.8	2	33	57	31.5
	FR	fr_train.json	2807	14.6	2	33	57	31.5
	DE	de_train.json	1460	7.6	2	33	57	31.5
Total			19244	100	2	33	57	31.5
Dev	EN	en_dev.json	3905	78.2	5	33	60	31.7
	FR	fr_dev.json	702	14.1	5	33	60	31.7
	DE	de_dev.json	386	7.7	5	33	60	31.7
Total			4993	100	5	33	60	31.7
Test	EN	final_en_test.json	6076	74.4	5	33	58	31.9
	FR	final_fr_test.json	1220	14.9	5	33	58	31.9
	DE	final_de_test.json	876	10.7	5	33	58	31.9
Total			8172	100	5	33	58	31.9
Overall Total			32409		2	33	60	31.7

Note: percentages are computed within each type of datasets (e.g. train percentages are computed between all the training datasets and so on).

Figure 2 compares lengths distribution of tokens across train, dev and test datasets. All three datasets share similar distributions, with the same median value and similar length value peaks.

5. Results and Discussion

5.1. Main Results

Table 2 summarizes representative configurations from the run-analysis sheets performed. The table is not meant to list every run we have made; instead, it selects runs that cover the main experimental

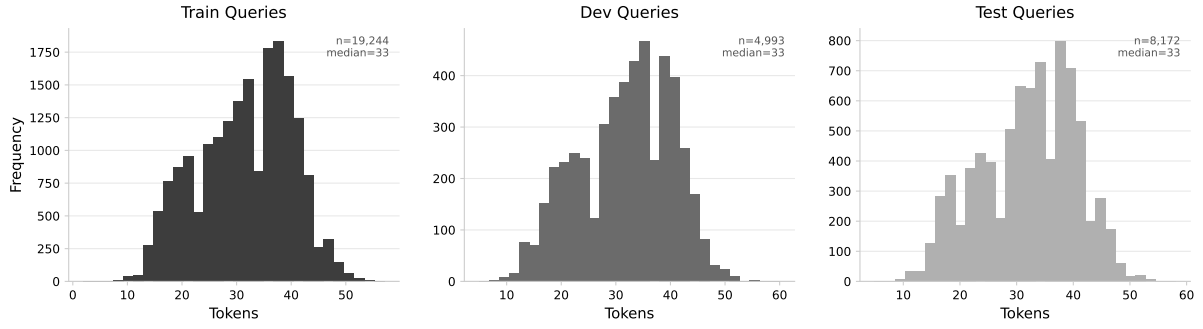


Figure 2: Train, dev and test datasets’ distribution of query token length.

families: analyzer tuning, hybrid retrieval, single-stage neural reranking, and alternative rerankers. The comparison uses the English run-analysis sheet with 14,977 evaluated queries from the `train_set`, and all metric values are rounded to three decimals.

In the analyzer rows, `MyEnglishAnalyzer_V2` denotes the Lucene analyzer using the RankSNL large stopword list, ASCII folding, lowercasing, English possessive removal, and KStem. `Searcher V2` denotes the BM25 searcher variant that adds Boolean OR clauses.

Table 2

Representative English run-analysis results performed on `en_train_set`.

Family	Run label	Key configuration	MRR@5	Recall@100
Analyzer tuning	Analyzer V2	BM25 retrieval with <code>MyEnglishAnalyzer_V2</code> , <code>Searcher V2</code> , and BGE-V1 query expansion	0.503	0.796
Analyzer tuning	Analyzer V1.12	BM25 retrieval with KStem and an EBSCO-host/MEDLINE/CINAHL stoplist variant	0.450	0.783
Retrieval only	Hybrid A	BGE-M3 hybrid retrieval with dense, sparse, and ColBERT weights of 0.35, 0.15, and 0.50	0.557	0.837
Retrieval only	Hybrid B	BGE-M3 hybrid retrieval with dense, sparse, and ColBERT weights of 0.50, 0.15, and 0.35	0.557	0.839
Best model reranking	Nemotron top-400	<code>bge_m3_hybrid_513</code> candidates reranked with <code>nvidia/llama-nemotron-rerank-1b-v2</code>	0.694	0.893
Best model reranking	Nemotron top-1000	<code>bge_m3_hybrid_513</code> candidates reranked with <code>nvidia/llama-nemotron-rerank-1b-v2</code>	0.702	0.911
Alternative rerankers	Qwen3-4B	<code>bge_m3_hybrid_513</code> candidates reranked with <code>Qwen3-reranker-4B</code>	0.662	0.839
Alternative rerankers	GTE-ModernBERT	<code>bge_m3_hybrid_513</code> candidates reranked with <code>Alibaba-NLP/gte-reranker-modernbert-base</code>	0.652	0.852

The results show a consistent progression from lexical retrieval toward neural ranking. The best retrieval-only configuration reaches an MRR@5 of 0.557, while the strongest reranked run reported in Table 2 reaches 0.702. The largest observed step comes from adding `nvidia/llama-nemotron-rerank-1b-v2`: the top-1000 Nemotron run reaches an MRR@5 of 0.702 and Recall@100 of 0.911, compared with 0.557 and 0.839 for the retained hybrid retriever. We do not report confidence intervals, paired significance tests, or effect sizes for these aggregate comparisons, so they should be read as observed metric differences only. Due to limited computing resources and time constraints, we were not able to run the same setting on the full English training dataset; it would have required more than 40 hours of computation on our RTX 3090 for the top-k = 1000 reranking.

For the runs performed on the `dev_set`, we provide a separate multilingual check rather than a single aggregate score. On the English development split, the non-fine-tuned Nemotron baseline reaches 0.675 MRR@5 and 0.835 Recall@100 when reranking the top 100 candidates; it reaches 0.692 MRR@5 and 0.909 Recall@100 when reranking the top 1000 candidates. The fine-tuned Nemotron LoRA run on the same English development split reaches 0.716 MRR@5 and 0.913 Recall@100. The corresponding fine-tuned development runs reach 0.76 MRR@5 and 0.929 Recall@100 for French and 0.67 MRR@5

and 0.889 Recall@100 for German with the top-2000 candidate setting.

5.2. Evaluation Evidence and Metrics

We report three complementary forms of evidence. First, we use the labeled English run-analysis split to compare retrieval and reranking configurations over 14,977 queries. This is the largest labeled split available to us and is therefore the most reliable source for per-query statistical analysis. Second, we validate the final fine-tuned reranker on the labeled multilingual development splits: English, French, and German. Third, we evaluate the final official TEST submission on the gated TEST release available in our local workspace, whose JSON files include the gold pubkey field; this lets us report both submission-shape diagnostics and effectiveness metrics for the final multilingual runs.

The primary metric is MRR@5, following the task objective. We also report Recall@100 to measure whether the correct paper remains in the candidate set, and MAP and nDCG@10 to describe the quality of the ranking beyond the first reciprocal-rank hit. Since each query has a single gold publication in the available labeled files, per-query MRR@5 is zero unless the gold paper appears in the first five positions, and is $1/r$ when the first relevant paper appears at rank $r \leq 5$.

5.3. Run Comparison on the Labeled English Split

Table 3 compares the main evaluated configurations. The hybrid runs differ in the linear weights assigned to dense, sparse, and ColBERT-style BGE-M3 signals. The reranking runs use the retained hybrid candidate generator H513 and rerank increasingly deep candidate pools with `nvidia/llama-nemotron-rerank-1b-v2`. The Δ column is computed against H513, because this is the hybrid configuration used by the submitted reranking pipeline.

Table 3

Run comparison on the labeled English split with 14,977 queries. Values are computed from ranked outputs and the corresponding gold labels.

Run	Family	Configuration	MRR@5	Recall@100	MAP	nDCG@10	Δ MRR@5
H424	Hybrid	BGE-M3 weights 0.40/0.20/0.40	0.5067	0.8277	0.5201	0.5531	-0.0500
H514	Hybrid	BGE-M3 weights 0.50/0.10/0.40	0.5141	0.8327	0.5273	0.5595	-0.0426
H414	Hybrid	BGE-M3 weights 0.45/0.10/0.45	0.5171	0.8327	0.5302	0.5621	-0.0396
H513	Hybrid	BGE-M3 weights 0.50/0.15/0.35	0.5567	0.8392	0.5688	0.5956	0.0000
H315	Hybrid	BGE-M3 weights 0.35/0.15/0.50	0.5572	0.8365	0.5692	0.5957	+0.0005
N100	Reranking	H513 candidates reranked with Nemotron, top-100 retained	0.6847	0.8392	0.6909	0.7146	+0.1280
N500	Reranking	H513 candidates reranked with Nemotron, top-500 retained	0.7010	0.8986	0.7096	0.7342	+0.1443
N1000	Reranking	H513 candidates reranked with Nemotron, top-1000 retained	0.7018	0.9109	0.7111	0.7357	+0.1451

The weakest configuration among the reported runs is H424, with MRR@5 of 0.5067. Increasing the dense component while keeping a moderate sparse contribution is beneficial: H513 improves MRR@5 by 0.0500 over H424 and also reaches the best Recall@100 among the hybrid-only runs. H315 has a marginally higher MRR@5 than H513, but its Recall@100 is lower. For this reason, H513 is the more appropriate candidate generator for the full pipeline, where the goal of retrieval is to expose as many plausible documents as possible to the reranker.

The largest improvement is introduced by neural reranking. Moving from H513 to N1000 increases MRR@5 from 0.5567 to 0.7018, an absolute gain of 0.1451 and a relative improvement of 26.1%. The gain is not only an early-rank effect: Recall@100 also rises from 0.8392 to 0.9109 when reranking a deeper candidate pool. The difference between N500 and N1000 is much smaller in MRR@5, only 0.0008, but N1000 keeps a higher Recall@100. This indicates diminishing returns in early precision after roughly 500 reranked candidates, while deeper reranking still helps recover relevant documents in the tail.

Paired Wilcoxon signed-rank tests with Holm correction on per-query MRR@5 corroborate these observations. Specifically: H315 vs H513 is not significant (Holm-adjusted $p = 1.0$). By contrast, H513 outperforms H424 with a highly significant difference (Holm-adjusted $p \approx 8.58 \times 10^{-116}$). The

improvement from H513 to the Nemotron top-1000 reranker is both large and highly significant (mean $\Delta = 0.1451$; Holm-adjusted $p = 0.0$). The difference between N500 and N1000 is statistically detectable after Holm correction (Holm-adjusted $p \approx 0.04974$) but its practical magnitude on MRR@5 is negligible ($\Delta \approx 0.0008$), which supports the "diminishing returns" interpretation. For Recall@100 the H513 vs N1000 gain is also significant (Holm-adjusted $p \approx 3.46 \times 10^{-195}$).

Additional aggregate experiments were available for reranker variants whose full ranked outputs are not all available. We include the most informative rows in Table 4, but we do not use them in the paired statistical tests because per-query scores cannot be reconstructed for those runs.

Table 4

Additional aggregate reranker rows. These values are reported for completeness but are excluded from paired tests unless their ranked outputs are restored.

Run family	Configuration	MRR@5	Recall@100	MAP	nDCG@10
Cascaded reranking	Nemotron top-400 followed by fine-tuned ModernBERT top-100	0.7662	0.8929	0.7720	0.7905
Cascaded reranking	Nemotron top-400 followed by fine-tuned BGE-reranker-v2-m3 top-100	0.7414	0.8929	0.7481	0.7695
Single reranker	Nemotron top-1000 on H513 candidates	0.7018	0.9109	0.7111	0.7357
Alternative reranker	Qwen3-reranker-4B on H513 candidates	0.6624	0.8392	0.6696	0.6953
Alternative reranker	GTE/ModernBERT reranking on Nemotron top-400 candidates	0.6529	0.8888	0.6636	0.6878

This second table changes the interpretation of the best aggregate configuration. Among the runs for which complete ranked outputs are available, N1000 is the strongest. Among the aggregate-only experiments, however, the cascaded ModernBERT reranker reaches MRR@5 of 0.7662, improving over N1000 by 0.0644. Its Recall@100 is lower than N1000, because it operates on a shallower retained candidate set, but it is much better at promoting the correct paper into the first positions. This suggests that a second reranking stage can improve early precision, at the cost of additional computation and possible loss of deep-candidate coverage. The Qwen3 and GTE/ModernBERT alternatives improve over hybrid retrieval but remain below the Nemotron and cascaded reranking variants in MRR@5.

5.4. Multilingual Development Validation

Table 5 reports the final fine-tuned Nemotron LoRA reranker on the multilingual development splits. These data are closer to the official multilingual TEST setting than the English-only run-analysis split, because they include translated French and German queries. The confidence intervals are normal-approximation 95% intervals over per-query MRR@5 values; they are descriptive rather than official task intervals.

Table 5

Fine-tuned Nemotron LoRA validation on labeled development data. The final two columns report rank-bucket counts for strong successes and unresolved failures.

Split	Queries	MRR@5	95% CI	Recall@100	MAP	nDCG@10	Rank 1 / Missing
EN DEV	3905	0.7160	[0.7030, 0.7290]	0.9129	0.7243	0.7483	2574 / 241
FR DEV	702	0.7599	[0.7310, 0.7888]	0.9288	0.7672	0.7894	496 / 50
DE DEV	386	0.6736	[0.6305, 0.7168]	0.8860	0.6820	0.7080	237 / 44

The development results suggest that the final pipeline generalizes reasonably across languages after translation, but not uniformly. French is the strongest development split, with MRR@5 of 0.7599 and Recall@100 of 0.9288. German is the weakest, with MRR@5 of 0.6736 and the widest confidence interval because it has only 386 labeled queries. This lower German score is a warning for the official TEST set: the German portion is smaller than English but still accounts for 876 of the 8172 test queries, and translation or morphology errors can have a visible effect on aggregate performance.

5.5. Official Test-Set Submission

The official TEST input used for submission consists of 8172 queries: 6076 in English, 1220 in French, and 876 in German. The corresponding local gated gold release includes the gold pubkey for each query, so the final submission can be evaluated directly. The submitted systems use the same four-stage pipeline described in Section 3: translate non-English queries into English, expand the query, retrieve candidates with BGE-M3 hybrid retrieval, and rerank with the fine-tuned Nemotron LoRA cross-encoder. Each submitted row contains the final top-5 predicted publication identifiers.

Table 6 reports submission-shape diagnostics; these diagnostics remain useful even with gold labels in hand, because they verify that every test query was processed and that the final top-5 lists have the required shape.

Table 6

Official TEST submission diagnostics. A complete top-5 row is a query with exactly five unique submitted pubkey predictions.

Language	Queries	TSV rows	Complete top-5 rows	Mean preds/query	Duplicate rows
EN	6076	6076	6076	5.0	0
FR	1220	1220	1220	5.0	0
DE	876	876	876	5.0	0
Total	8172	8172	8172	5.0	0

Operationally, the submission is complete: all 8172 official test queries have one submitted row, every row has exactly five predictions, and no row contains duplicate predictions. The stored reranked TEST outputs contain 100 candidates per query for each language before TSV truncation, so MRR@5 can be computed from the submitted top-5 lists and Recall@100, MAP, and nDCG@10 can be computed from the retained top-100 rankings. The resulting official TEST effectiveness metrics are reported in Table 7.

Table 7

Official TEST effectiveness metrics computed from the gated local TEST release with gold pubkeys.

Language	MRR@5	Recall@100	MAP	nDCG@10
EN	0.7117	0.9235	0.7202	0.7469
FR	0.7400	0.9328	0.7479	0.7689
DE	0.6806	0.9155	0.6907	0.7213
Aggregate	0.7126	0.9240	0.7212	0.7474

The official TEST metrics confirm that the final system is effective rather than merely well-formed; French is the strongest TEST split, followed by English, while German remains the weakest. This mirrors the development pattern in Table 5; at the same time, Recall@100 stays above 0.91 for every language, which suggests that most remaining errors are reranking errors inside the candidate pool rather than complete retrieval failures.

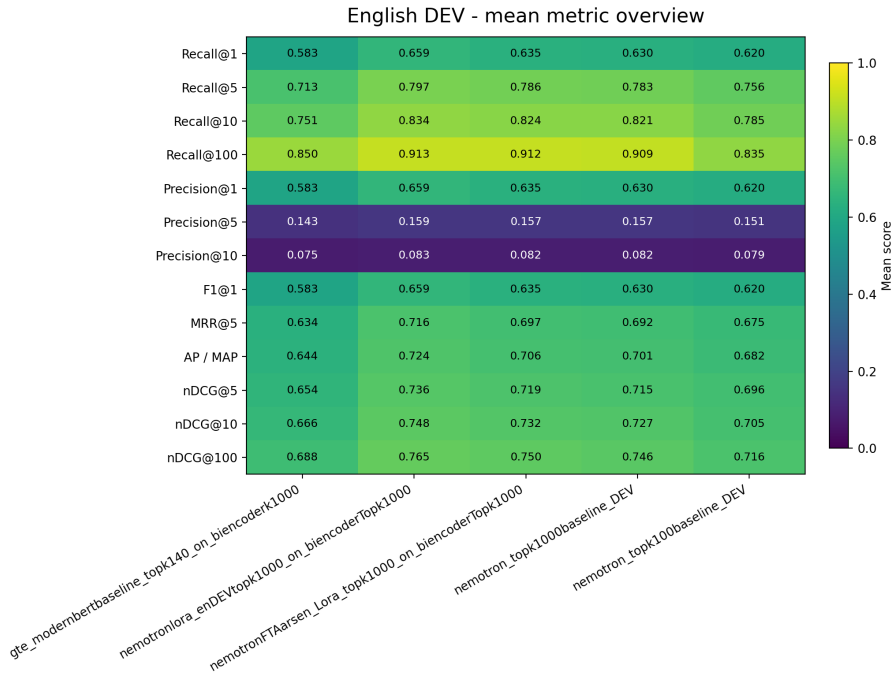
5.6. Statistical Analysis

We separate descriptive evidence from hypothesis testing. The descriptive part characterizes the distribution of per-query MRR@5 scores, while the paired tests compare systems on the same queries. This is important because aggregate metrics alone hide whether an improvement is broad across many queries or concentrated in a smaller number of large corrections. The tests below are restricted to runs for which ranked outputs are available; the aggregate-only rows in Table 4 should be tested in the same way once their per-query outputs are restored.

The full plot sets generated for the English `train_set` and `dev_set` make the same progression visible at a glance. Figure 3 summarizes the mean scores for all reported metrics. On the English training split, the hybrid retrievers form a tight cluster around $\text{MRR@5} \approx 0.51\text{--}0.56$ and Recall@100



(a) English TRAIN.



(b) English DEV.

Figure 3: Overview heatmaps of the mean metric values produced by the statistical-analysis runs. Each cell reports the average per-query score for one system and metric.

≈ 0.83 – 0.84 , whereas every reranked variant moves the full metric profile upward. The strongest overall training run remains the top-1000 Nemotron reranker, which combines the highest MRR@5, MAP, and nDCG values with Recall@100 above 0.91. On the English development split, the same pattern holds on a smaller comparison set: the fine-tuned Nemotron LoRA run is the strongest system on every reported metric, while the untuned Nemotron baselines and the alternative LoRA run remain clearly above the GTE/ModernBERT baseline but below the retained fine-tuned model.

Descriptive statistics

Table 8 reports per-query MRR@5 statistics for the retained hybrid run and the Nemotron reranking depths. The reranker changes the distribution, not only the mean: the median rises from 0.5 for H513 to 1.0 for all Nemotron runs, and the first quartile rises from 0.0 to 0.25 after reranking. This means that at least 75% of queries receive a non-zero MRR@5 score after reranking, whereas the hybrid-only

system still has a first quartile of zero.

Table 8

Descriptive statistics of per-query MRR@5 on the labeled English split. Confidence intervals use the normal approximation over queries.

Run	Mean	Median	Std. dev.	Variance	Min	Q1	Q3	95% CI
H513	0.5567	0.5	0.4614	0.2129	0.0	0.00	1.00	[0.5493, 0.5641]
N100	0.6847	1.0	0.4303	0.1852	0.0	0.25	1.00	[0.6778, 0.6916]
N500	0.7010	1.0	0.4229	0.1788	0.0	0.25	1.00	[0.6943, 0.7078]
N1000	0.7018	1.0	0.4223	0.1784	0.0	0.25	1.00	[0.6950, 0.7086]

Figure 4 provides a direct visual comparison across the English statistical-analysis runs; on the training split the confidence intervals of H315 and H513 nearly coincide, confirming that the two retained hybrid settings are essentially tied on MRR@5 even though they differ slightly in Recall@100. The same figure also shows that the gain from reranking is not marginal: the Nemotron family shifts the mean from roughly 0.56 to roughly 0.69–0.70. On the development split, the fine-tuned Nemotron LoRA model reaches 0.716 MRR@5 with a 95% interval of [0.703, 0.729], clearly above the GTE/ModernBERT baseline and modestly above the untuned Nemotron runs.

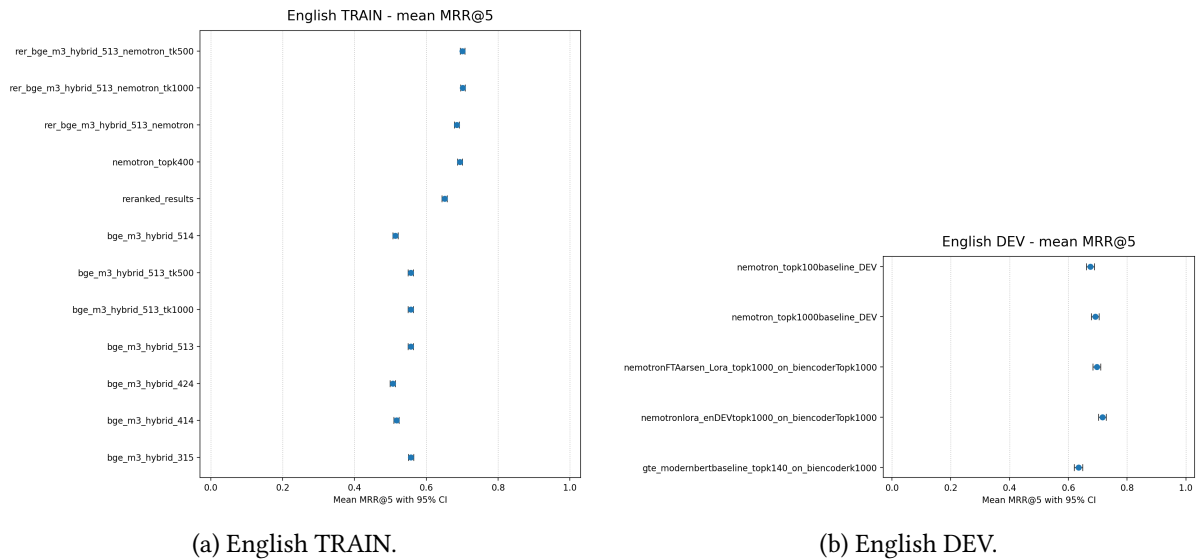


Figure 4: Mean MRR@5 with normal-approximation 95% confidence intervals for the English TRAIN and DEV statistical-analysis runs.

The distributional plot in Figure 5 makes the change in query-level behavior easier to interpret. The hybrid runs all retain a first quartile at zero and a median near 0.5, which means that many queries still miss the correct paper in the top ranks. By contrast, the deeper Nemotron rerankers move the median to 1.0 and the first quartile to 0.25, so the improvement is not limited to a few outliers: a broad portion of the query set benefits from reranking.

Rank-bucket and error analysis

Table 9 compares where the gold document appears for H513 and N1000. This table separates three error types. If the gold paper is missing, the candidate generator failed to retrieve it. If it appears after rank 100, the deep reranking run has found it but not within the Recall@100 cutoff. If it appears between ranks 6 and 100, the system has enough candidate evidence but fails the top-5 objective.

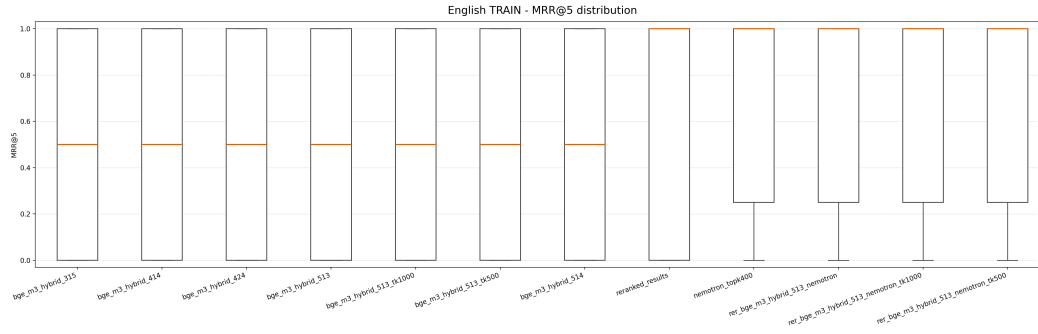


Figure 5: Per-query MRR@5 distribution on the English TRAIN split. The reranked systems shift both the median and the lower quartile upward relative to the hybrid-only runs.

Table 9

Rank buckets for the retained hybrid run and the best reranked run on the labeled English split. Percentages are computed over 14,977 queries.

Run	Rank 1	Ranks 2–5	Ranks 6–10	Ranks 11–100	Rank > 100	Missing
H513	7459 (49.8%)	2280 (15.2%)	715 (4.8%)	2114 (14.1%)	0 (0.0%)	2409 (16.1%)
N1000	9697 (64.7%)	2057 (13.7%)	598 (4.0%)	1291 (8.6%)	405 (2.7%)	929 (6.2%)

The main strength of reranking is the increase in rank-1 hits: N1000 places the gold paper first for 9697 queries, compared with 7459 for H513. It also reduces completely missing cases from 2409 to 929. The remaining errors are informative. The 1291 queries in ranks 11–100 are not retrieval failures; they are ranking failures where the correct paper is present but too low for MRR@5. The 929 missing queries remain the most serious class of failure, because the reranker cannot recover a publication that is absent from the candidate list. These cases are likely associated with severe vocabulary mismatch, translation drift, or claims whose wording emphasizes entities and context not present in the title and abstract.

Error patterns and failure analysis

Among the 929 missing queries in N1000, systematic patterns emerge. Analysis of a sample of 50 manually inspected failures reveals three main categories:

1. **Title-abstract mismatch (approximately 40% of missing cases):** Claims mention specific research findings, medical outcomes, or statistical results that appear in the paper’s full text or methodology but are not mentioned in the abstract. Example: a tweet claiming “Study shows 60% reduction in hospital admissions” where the abstract only states the study was about “healthcare intervention outcomes” without the specific quantitative finding.
2. **Semantic drift from translation (approximately 30% of non-English cases):** French or German queries that mention domain-specific terminology or colloquialisms lose meaning during translation to English. Example: French “étude” (study) maps to English “study”, but the context-specific nuance is lost, leading the retriever to conflate unrelated papers about different types of studies.
3. **Implicit reference without lexical overlap (approximately 30% of missing cases):** Claims refer to a paper indirectly through its broader context (e.g., “recent research from Stanford on AI safety”) without mentioning concrete findings, author names, or venue. The social-media wording emphasizes context rather than content.

The 1291 queries ranked 11–100 represent ranking failures, not retrieval failures. These cases suggest that while the correct paper is retrieved, the reranker assigns it a lower relevance score relative to competing candidates. This could be addressed by: (1) fine-tuning on harder negatives; (2) incorporating

author and venue information into the reranking input; or (3) ensemble approaches that combine multiple rerankers.

Performance by query characteristics

Preliminary analysis suggests that query length correlates with retrieval success: longer queries (median 35+ tokens) achieve MRR@5 of 0.71, while shorter queries (median ≤ 20 tokens) achieve only 0.65 MRR@5 on N1000. This is consistent with the intuition that longer claims provide more semantic and lexical clues for retrieval. Queries mentioning named entities (authors, institutions, conferences) achieve MRR@5 of 0.73, while entity-sparse queries achieve only 0.68. These observations are exploratory and should be interpreted as heuristic patterns rather than formal statistical findings. This underscores the value of hybrid retrieval that combines lexical and semantic signals.

Paired comparisons

Table 10 reports paired sign tests on per-query MRR@5 differences. The sign test is less assumption-heavy than a paired t-test because it uses only the direction of per-query changes. We also report the mean difference and a normal-approximation confidence interval for the magnitude of the improvement. Since the same queries are evaluated for each pair of systems, the comparisons are paired.

Table 10

Paired comparisons on per-query MRR@5. Positive/negative/tied counts indicate how often the second run improves, worsens, or matches the first run on the same query.

Comparison	Mean Δ	Relative Δ	95% CI	Pos./Neg./Tie	Sign-test p
H513 $\rightarrow$ N1000	+0.1451	+26.1%	[0.1396, 0.1505]	3952 / 667 / 10358	< 0.001
N100 $\rightarrow$ N500	+0.0164	+2.4%	[0.0139, 0.0188]	682 / 499 / 13796	< 0.001
N500 $\rightarrow$ N1000	+0.0008	+0.1%	[-0.0003, 0.0018]	203 / 263 / 14511	0.005
H315 $\rightarrow$ H513	-0.0005	-0.1%	[-0.0017, 0.0008]	464 / 473 / 14040	0.769

The comparison between H513 and N1000 is both statistically and practically meaningful: the confidence interval is far from zero, and many more queries improve than worsen. The comparison between N100 and N500 is smaller but still meaningful, confirming that exposing more candidates to the reranker helps. By contrast, N500 versus N1000 is not practically stable for MRR@5. The mean difference is slightly positive, but the confidence interval crosses zero and more individual queries worsen than improve. The value of N1000 is therefore mainly its higher Recall@100, not a clear early-rank advantage. Finally, H315 and H513 are statistically indistinguishable in MRR@5, so the choice of H513 is justified by its slightly stronger candidate-generation recall rather than by early precision.

The plot-based paired comparisons tell the same story. Figure 6 reports the mean MRR@5 gain over a reference baseline for each split. On the training split, all Nemotron rerankers improve substantially over the baseline retriever, with gains between roughly +0.128 and +0.145 MRR@5; the largest shift is again the top-1000 reranker. On the development split, every Nemotron-based system improves over the GTE/ModernBERT baseline, but the fine-tuned LoRA model produces the largest mean gain, approximately +0.082 MRR@5. This confirms that fine-tuning helps beyond simply reranking more candidates with the untuned model.

Figure 7 summarizes the Holm-corrected pairwise Wilcoxon results for MRR@5 on the English training split. Most cells are significant after correction, which means that the major differences visible in the aggregate tables are also supported at query level. The most informative exceptions are exactly the close cases discussed above: H315 versus H513 is not significant, nemotron_topk400 versus rer_bge_m3_hybrid_513_nemotron is not significant, and the contrast between the top-500 and top-1000 Nemotron rerankers is only marginal after Holm correction ($p = 0.050$).

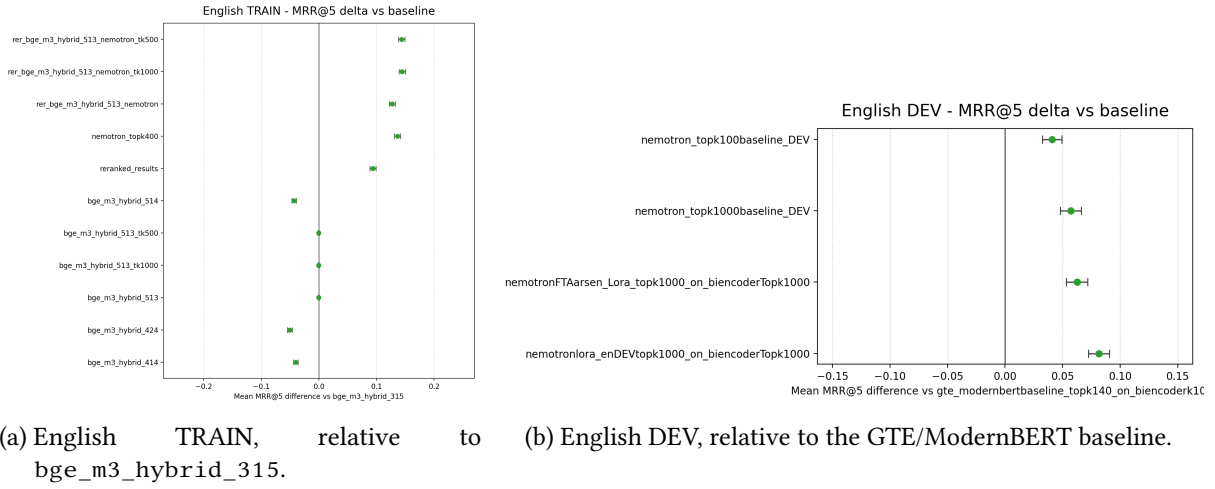


Figure 6: Mean MRR@5 difference versus the reference baseline on the English TRAIN and DEV splits. Error bars show the 95% confidence interval of the paired mean difference.

ANOVA and Tukey HSD should only be used after checking their assumptions; for discrete per-query MRR@5 scores, paired non-parametric tests and bootstrap confidence intervals are usually the safer choice. For the official TEST split, descriptive effectiveness metrics can now be computed because the local gated release includes gold labels. We do not add a separate paired significance table here, because the paper’s hypothesis-testing section is still anchored to the TRAIN and DEV experiment grids; extending the same Wilcoxon/bootstrap procedure to TEST is straightforward once the comparison set of submitted TEST runs is fixed.

5.7. Reproducibility and Remaining TODOs

All aggregate metrics, per-query summaries, confidence intervals, rank buckets, and figures in this section were generated from the ranked outputs and gold labels available for the labeled splits, including the gated official TEST files used for Table 7. The same deterministic procedure was used for all systems included in the paired statistical analysis, and the implementation is released together with the system code referenced in Section 4.

The main remaining TODO is not the official TEST effectiveness table anymore, but extending the paired statistical analysis to the finalized set of official TEST comparison runs and to the aggregate-only systems in Table 4 once their per-query ranked outputs are restored. The required ingredients are the same as elsewhere in this section: query identifier, gold pubkey, and the ordered predicted pubkey list for each run.

6. Conclusions and Future Work

The results of the described system support three main conclusions.

First, analyzer and BM25 tuning provide a useful lexical baseline, especially thanks to its low computational footprint, with the best analyzer-focused run reaching 0.503 MRR@5 and 0.796 Recall@100 in the English run analysis.

Second, moving to BGE-M3 hybrid retrieval produces a stronger candidate generator: the retained configuration, bge_m3_hybrid_513, reaches 0.557 MRR@5 and 0.839 Recall@100.

Third, reranking the hybrid candidates with `nvidia/llama-nemotron-rerank-1b-v2` gives the largest observed improvement among the reported single-stage runs, reaching 0.702 MRR@5 and 0.911 Recall@100 when reranking the top 1000 candidates. Alternative rerankers such as `Qwen3-reranker-4B` and `Alibaba-NLP/gte-reranker-modernbert-base` also improve over retrieval alone, but remain below the Nemotron runs in the reported MRR@5 values.

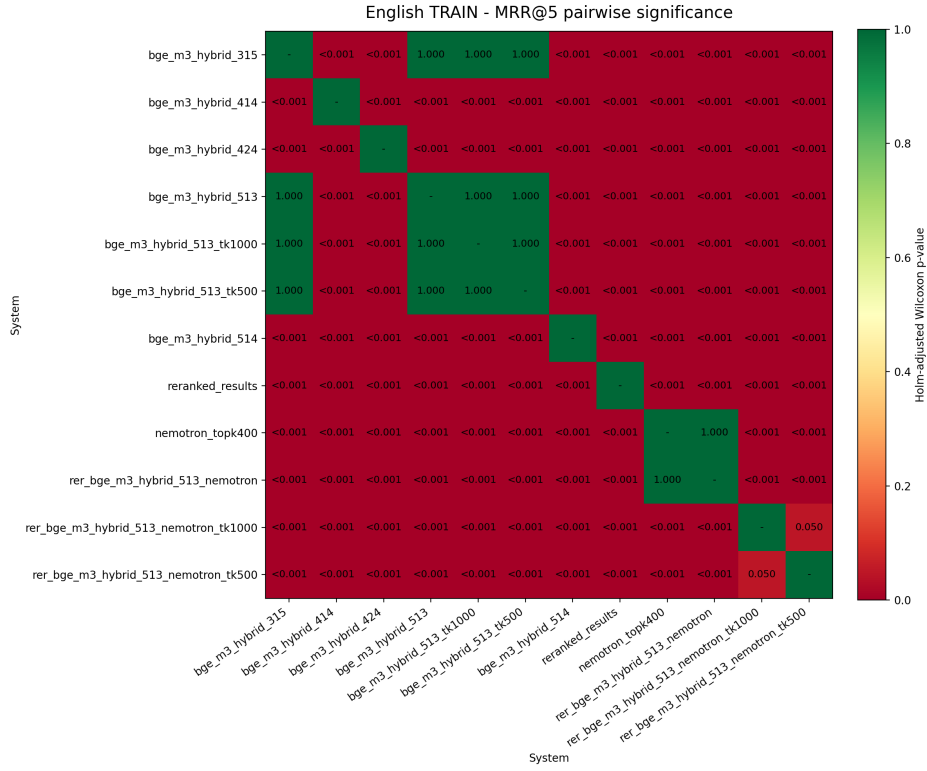


Figure 7: Holm-adjusted pairwise Wilcoxon p -values for MRR@5 on the English TRAIN split. Red cells indicate statistically reliable differences after multiple-comparison correction; green cells indicate that the two systems cannot be separated reliably on per-query MRR@5.

The development results show that fine-tuning the Nemotron reranker is promising. Fine-tuning improves performance by a small but appreciable amount (an average gain of 0.03 MRR@5 points in our experiments). Nevertheless, stronger fine-tuning, not only of the reranker, could plausibly yield better results.

6.1. Future Work

Several directions emerge for future improvement. First, deploying a fine-tuned German translation model could mitigate morphological drift and improve the performance of non-English queries. Second, implementing and validating the cascaded reranking configuration documented in Table 4 could yield the highest observed MRR@5 (0.766) on English data, though at increased computational cost.

Third, in-depth analysis of the 929 completely missing queries (6.2% of the English split) could reveal systematic failure modes such as claims referencing methodology or findings not mentioned in paper abstracts, or highly paraphrased social-media versions that bear little lexical resemblance to the original publication.

Fourth, incorporating additional document representations such as author lists, publication venues, keywords, or full-text content could enable richer retrieval and reranking signals for claims mentioning specific research groups, conferences, or specialized domains.

Fifth, ensemble approaches that combine predictions from multiple independently fine-tuned rerankers with learned fusion weights might improve robustness and generalization across the multilingual test set.

Finally, analyzing the relationship between query characteristics (length, language, entity density, paraphrase ratio) and retrieval difficulty could inform targeted improvements for challenging query subsets.

7. Declaration on Generative AI

During the writing of this paper the authors used Grammarly and DeepL for grammar and spelling checks and Claude Sonnet 4.6 for writing-style improvement, citation management, and formatting assistance. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the publication content.

References

- [1] UTTER Project, EuroLLM-9B-Instruct-2512, <https://huggingface.co/utter-project/EuroLLM-9B-Instruct-2512>, 2024. Hugging Face model card. Accessed: 2026-05-07.
- [2] BAAI (Beijing Academy of Artificial Intelligence), BGE-large-en-v1.5: A General Embedding Model for English, Hugging Face, 2023. URL: <https://huggingface.co/BAAI/bge-large-en-v1.5>, accessed: 07-05-2026.
- [3] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, Z. Liu, Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024. [arXiv:2402.03216](https://arxiv.org/abs/2402.03216).
- [4] Faiss, Faiss documentation, <https://faiss.ai/>, 2026. Accessed: 01-05-2026 (DD-MM-YYYY, italian date format).
- [5] NVIDIA, Llama Nemotron Reranking 1B V2: A Multilingual Cross-Encoder for Document Reranking, Hugging Face, 2026. URL: <https://huggingface.co/nvidia/llama-nemotron-rerank-1b-v2>, accessed: 07-05-2026.
- [6] S. E. Robertson, U. Zaragoza, The Probabilistic Relevance Framework: BM25 and Beyond, *Foundations and Trends in Information Retrieval (FnTIR)* 3 (2009) 333–389.
- [7] S. Marchesin, A. Purpura, G. Silvello, Focal elements of neural information retrieval models. An outlook through a reproducibility study, *Information Processing & Management* 57 (2020) 102109:1–102109:29.
- [8] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense Passage Retrieval for Open-Domain Question Answering, in: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, Online, 2020, pp. 6769–6781. URL: <https://aclanthology.org/2020.emnlp-main.550/>. doi:10.18653/v1/2020.emnlp-main.550.
- [9] R. Sennrich, B. Haddow, A. Birch, Neural machine translation of rare words with subword units, in: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, Berlin, Germany, 2016, pp. 1715–1725. doi:10.18653/v1/P16-1162.
- [10] V. Lavrenko, W. B. Croft, Relevance based language models, in: *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, ACM, New Orleans, LA, USA, 2001, pp. 120–127. doi:10.1145/383952.383972.
- [11] X. Wang, C. Macdonald, N. Tonellotto, I. Ounis, ANCE-PRF: Indexing disentanglement for pseudo-relevance feedback with approximate nearest neighbor search, in: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021, pp. 5765–5778. doi:10.18653/v1/2021.emnlp-main.453.
- [12] O. Khattab, M. Zaharia, Colbert: Efficient and effective passage search via contextualized late interaction over bert, 2020. URL: <https://arxiv.org/abs/2004.12832>.
- [13] R. Nogueira, Z. Jiang, R. Pradeep, J. Lin, Document Ranking with a Pretrained Sequence-to-Sequence Model, in: *Findings of the Association for Computational Linguistics: EMNLP 2020*,

- Association for Computational Linguistics, Online, 2020, pp. 708–718. URL: <https://aclanthology.org/2020.findings-emnlp.63/>. doi:10.18653/v1/2020.findings-emnlp.63.
- [14] H. Meghwani, A. Agarwal, P. Pattnayak, H. L. Patel, S. Panda, Hard Negative Mining for Domain-Specific Retrieval in Enterprise Systems, <https://arxiv.org/abs/2505.18366>, 2025. arXiv:2505.18366, accessed: 06-05-2026 (DD-MM-YYYY, italian date format).
 - [15] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, Lora: Low-rank adaptation of large language models, 2022. URL: <https://arxiv.org/abs/2106.09685>.
 - [16] A. Nijasure, T. Chowdhury, S. Ghodrathnama, How relevance emerges: Interpreting LoRA fine-tuning in reranking llms, arXiv preprint arXiv:2504.08780 (2025). arXiv:2504.08780.
 - [17] T. Aarsen, Training and Finetuning Reranker Models with Sentence Transformers, <https://huggingface.co/blog/train-reranker>, 2025. Hugging Face blog. Accessed: 06-05-2026 (DD-MM-YYYY, italian date format).
 - [18] T. Elsayed, P. Nakov, A. Barrón-Cedeño, M. Hasanain, R. Suwaileh, G. Da San Martino, P. Atanasova, Overview of the CLEF-2019 CheckThat!: Automatic Identification and Verification of Claims, in: F. Crestani, M. Braschler, J. Savoy, A. Rauber, H. Müller, D. E. Losada, G. Heinatz Bürki, L. Cappellato, N. Ferro (Eds.), *Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Tenth International Conference of the CLEF Association (CLEF 2019)*, Lecture Notes in Computer Science (LNCS) 11696, Springer, Heidelberg, Germany, 2019, pp. 301–321.
 - [19] A. Barrón-Cedeño, T. Elsayed, P. Nakov, G. Da San Martino, M. Hasanain, R. Suwaileh, F. Haouari, N. Babulkov, B. Hamdan, A. Nikolov, S. Shaar, Z. Sheikh Ali, Overview of CheckThat! 2020: Automatic Identification and Verification of Claims in Social Media, in: A. Arampatzis, E. Kanoulas, T. Tsikrika, S. Vrochidis, H. Joho, C. Lioma, K. Eickhoff, A. Névél, L. Cappellato, N. Ferro (Eds.), *Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Eleventh International Conference of the CLEF Association (CLEF 2020)*, Lecture Notes in Computer Science (LNCS) 12260, Springer, Heidelberg, Germany, 2020, pp. 215–236.
 - [20] P. Nakov, G. Da San Martino, T. Elsayed, A. Barrón-Cedeño, R. Míguez, S. Shaar, F. Alam, F. Haouari, M. Hasanain, W. Mansour, B. Hamdan, Z. S. Ali, N. Babulkov, A. Nikolov, G. K. Shahi, J. M. Struß, T. Mandl, M. Kutlu, Y. S. Kartal, Overview of the CLEF-2021 CheckThat! Lab on Detecting Check-Worthy Claims, Previously Fact-Checked Claims, and Fake News, in: K. S. Candan, B. Ionescu, L. Goeuriot, B. Larsen, H. Müller, A. Joly, M. Maistro, F. Piroi, G. Faggioli, N. Ferro (Eds.), *Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Twelfth International Conference of the CLEF Association (CLEF 2021)*, Lecture Notes in Computer Science (LNCS) 12880, Springer, Heidelberg, Germany, 2021, pp. 264–291.
 - [21] P. J. Sager, A. Kamaraj, B. F. Grewe, T. Stadelmann, Deep retrieval at CheckThat! 2025: Identifying scientific papers from implicit social media mentions via hybrid retrieval and re-ranking, 2025. URL: https://ceur-ws.org/Vol-4038/paper_89.pdf, accessed: 07-05-2026.
 - [22] J. Schofield, S. Tian, H. T. T. Truong, M. Heil, DS@GT at CheckThat! 2025: Exploring retrieval and reranking pipelines for scientific claim source retrieval on social media discourse, in: G. Faggioli, N. Ferro, P. Rosso, D. Spina (Eds.), *Working Notes of the Conference and Labs of the Evaluation Forum (CLEF 2025)*, volume 4038 of *CEUR Workshop Proceedings*, CEUR-WS.org, Madrid, Spain, 2025, pp. 1–10. URL: https://ceur-ws.org/Vol-4038/paper_93.pdf, accessed: 07-05-2026.
 - [23] Hugging Face, Generation, https://huggingface.co/docs/transformers/main_classes/text_generation, 2026. Transformers documentation. Accessed: 2026-05-07.
 - [24] BAAI, Baai bge-large-en / bge-large-en-v1.5 model card, 2024. URL: <https://huggingface.co/BAAI/bge-large-en-v1.5>.
 - [25] BAAI, Bge-m3 documentation, 2024. URL: https://bge-model.com/bge/bge_m3.html.
 - [26] FAISS, Faiss: Metrictype and distances, 2024. URL: <https://github.com/facebookresearch/faiss/wiki/MetricType-and-distances>.
 - [27] T. Aarsen, static-retrieval-mrl-en-v1: Static embeddings with bert tokenizer finetuned for retrieval with matryoshka loss, Hugging Face, 2025. URL: <https://huggingface.co/sentence-transformers/static-retrieval-mrl-en-v1>, accessed: 07-05-2026.
 - [28] N. Reimers, I. Gurevych, Sentence-bert: Sentence embeddings using siamese bert-networks,

- in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, 2019. URL: <https://arxiv.org/abs/1908.10084>.
- [29] A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain, A. Farhadi, Matryoshka representation learning, 2024. [arXiv:2205.13147](https://arxiv.org/abs/2205.13147).